

CS224N | Assignment 3

Selene · October 1, 2026

1 Problem 1

(a)

1. Make $k_j^\top q$ much larger than every other $k_i^\top q$.
2. The output is approximately v_j .

(b)

Let $q = \lambda(k_a + k_b)$, where λ is a sufficiently large positive number. Since the keys are pairwise orthogonal unit vectors,

$$k_a^\top q = k_b^\top q = \lambda.$$

For every $i \neq a, b$, we also have $k_i^\top q = 0$. Thus the softmax scores of k_a and k_b are both e^λ , while every other score is e^0 . When λ is large, the contributions of the other keys become negligible, so

$$c = \sum_i \alpha_i v_i \approx \frac{1}{2}(v_a + v_b).$$

(c)

1. The idea is the same as in (b), except that we use the mean of each key distribution rather than a particular sampled key. The perturbations are small, so the relevant keys remain close to μ_a and μ_b .
2. Here k_a varies substantially in length along the direction of μ_a . Write $k_a \approx s\mu_a$, where s fluctuates considerably. Then $k_a^\top q \approx \lambda s$, whereas the length of k_b is relatively stable and $k_b^\top q \approx \lambda$. Consequently, the attention output c shifts between v_a and v_b . In the first case its variance is small and the output is stable; in this case its variance is larger.

(d)

1. Unlike (c), where one head uses a single query to attend to both a and b , the two heads now divide the work. The means μ_i are pairwise orthogonal and the key perturbations are small, so q_1 has a large dot product only with k_a , giving $c_1 \approx v_a$. Similarly, $c_2 \approx v_b$. Therefore,

$$c = \frac{1}{2}(c_1 + c_2) \approx \frac{1}{2}(v_a + v_b).$$

2. This is the multi-head counterpart of (c)(ii). Changes in the length of k_a make $k_a^\top q_1$ fluctuate, so c_1 has relatively high variance: when k_a is long, $c_1 \approx v_a$, and when it is short, the attention paid to v_a weakens. But $q_2 \perp \mu_a$, so variation in k_a along μ_a has almost no effect on the second head. We still have $c_2 \approx v_b$ with low variance. Hence

$$c = \frac{1}{2}(c_1 + c_2).$$

Only the first head's output fluctuates; the second remains stable. In the single-head attention of (c)(ii), the weights on both v_a and v_b fluctuate together. Averaging the two heads therefore gives an output with lower variance.

(e)

Multi-head attention splits a task that one head would otherwise perform alone: one head attends to v_a , and another attends to v_b . A fluctuation in the length of one key mainly affects its corresponding head, while the other head remains stable. Averaging the head outputs reduces the variance of the final result and makes attention more robust.

2 Problem 2

(a)

1. From $X_{\text{perm}} = PX$, we obtain $Q_{\text{perm}} = PQ$, $K_{\text{perm}} = PK$, and $V_{\text{perm}} = PV$. Therefore,

$$\frac{Q_{\text{perm}} K_{\text{perm}}^\top}{\sqrt{d}} = P \frac{Q K^\top}{\sqrt{d}} P^\top.$$

Using the stated property of row-wise softmax and $P^\top P = I$,

$$\begin{aligned} H_{\text{perm}} &= \text{softmax}\left(P \frac{QK^\top}{\sqrt{d}} P^\top\right) PV \\ &= P \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) P^\top PV \\ &= PH. \end{aligned}$$

Since $P\mathbf{1} = \mathbf{1}$, it follows that

$$\begin{aligned} Z_{\text{perm}} &= \text{ReLU}(PHW_1 + \mathbf{1}b_1)W_2 + \mathbf{1}b_2 \\ &= \text{ReLU}(P(HW_1 + \mathbf{1}b_1))W_2 + P\mathbf{1}b_2 \\ &= P \text{ReLU}(HW_1 + \mathbf{1}b_1)W_2 + P\mathbf{1}b_2 \\ &= PZ. \end{aligned}$$

2. Without positional embeddings, the Transformer is permutation-equivariant: $Z_{\text{perm}} = PZ$. It cannot tell where a token originally appeared. Because the meaning of text depends on word order, positional embeddings are needed to supply position information.

(b)

1. Yes. With positional embeddings, $X_{\text{pos}} = X + \Phi$. If the tokens are permuted while their positions stay fixed, the new input is $X_{\text{perm, pos}} = PX + \Phi$. Permuting the original full input instead would give $P(X + \Phi) = PX + P\Phi$. These expressions generally differ, so different word orders produce different input representations. The Transformer can use the position assigned to each token to recover order information.

2. No, for the sinusoidal embeddings in the question. If distinct integer positions $t \neq s$ had identical embeddings, their first two coordinates would require $\sin t = \sin s$ and $\cos t = \cos s$. This implies $t = s + 2\pi k$ for some integer k . Since $t - s$ is an integer and $2\pi k$ can be an integer only when $k = 0$, we must have $t = s$, a contradiction.

3 Problem 3

(a)

This problem implements a decoder-only, GPT-2-style Transformer.

MLP. The MLP independently applies two linear transformations to each token, with GELU between them:

$$\text{MLP}(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2.$$

The first layer expands the hidden dimension from d_{model} to $4d_{\text{model}}$; the second projects it back to d_{model} .

CausalAttention. First obtain Q , K , and V from the input, then compute scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V.$$

Here M is a causal mask that permits the lower triangular part of the score matrix. Position t can attend only to positions $0, \dots, t$. Setting the scores for future positions to $-\infty$ makes their softmax weights zero. For multi-head attention, reshape $[B, T, d_{\text{model}}]$ into $[B, h, T, d_h]$, compute attention separately for each head, and concatenate the results.

DecoderBlock. This implementation uses Pre-LN: apply layer normalization before attention and the MLP, then add each sublayer's residual connection:

$$\begin{aligned} H &= X + \text{CausalAttention}(\text{LN}_1(X)), \\ Y &= H + \text{MLP}(\text{LN}_2(H)). \end{aligned}$$

Transformer. Add token and position embeddings, pass the result through the decoder blocks, then apply a final layer normalization and the language-model head to obtain logits:

$$\begin{aligned} H^{(0)} &= E_{\text{token}}(x) + E_{\text{position}}(0, \dots, T-1), \\ \text{logits} &= H^{(L)} W_{\text{lm}}^\top. \end{aligned}$$

At each generation step, take the token with the largest logit at the final position:

$$x_{\text{next}} = \arg \max_v \text{logits}_{-1,v}.$$

Append that token to the input and repeat. If the sequence exceeds the context length, keep only the most recent `context_length` tokens for the forward pass.

(b) Training the Transformer

`get_loss_on_batch` uses next-token prediction. Given an input sequence

$$(x_0, x_1, \dots, x_{T-1}),$$

the logits at position t predict x_{t+1} . Thus we remove the final logit position and the first label token:

$$\text{loss} = \text{CrossEntropy}(\text{logits}_{[:,0:T-1,:]}, x_{[:,1:T]}).$$

In the implementation, flatten the batch and sequence dimensions before passing them to `F.cross_entropy`. The final position has no next token, so it does not contribute to the loss.

After 100 batches, the loss decreases overall from about 10.83 to about 10.77, showing that the model has begun to learn from the data. The gradient norm stays within a modest range, with no obvious gradient explosion.

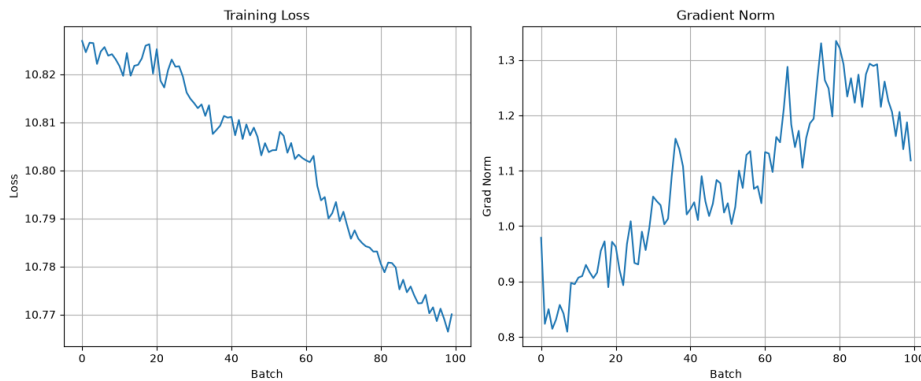


Figure 1: Training loss and gradient norm over 100 batches.

(c) Bonus: Faster Training

With the number of training steps fixed at 100, the goal is to obtain a lower final loss than the baseline under the same budget. I applied three changes in sequence: a higher learning rate, no weight decay, and a learnable bias in the language-model head.

Setting	Final loss after 100 steps
Baseline	≈ 10.77
Learning rate: 10^{-3}	6.449274
+ No weight decay	6.446831
+ LM head bias	6.427308

Increasing the learning rate produces the largest improvement in convergence over the fixed number of steps. Removing weight decay slightly improves the training loss under this budget; the LM-head bias can directly learn an overall frequency offset for each token and reduces the final loss further.